Trabajo Práctico Integrador

Expensa House

Búsqueda de productos con modelos preentrenados y gestión del hogar

Estudiante: Griselda Blanco
Materia: Seminario de Actualización
Institución: IFTS N.º 18
Revisión: 25 de octubre de 2026

A. El problema

Organizar las compras del hogar implica saber qué queda en la alacena y reconocer cuándo dos publicaciones describen un producto equivalente. Los nombres varían entre supermercados: una consulta puede decir “leche entera de un litro” y una oferta, “leche entera 1L”. El objetivo académico es acotado: evaluar si un modelo ya entrenado ayuda a recuperar productos relacionados con la consulta.

Las reglas son útiles para convertir unidades, exigir una marca o descartar una presentación distinta. Un modelo de similitud aporta flexibilidad ante paráfrasis y otros idiomas. No reemplaza las comprobaciones de precio, tamaño, vigencia ni disponibilidad. Por eso la aplicación combina criterios de negocio con evidencia experimental, sin presentar la similitud como garantía de equivalencia.

Alcance: el experimento utiliza un CSV de demostración. Sus precios no representan ofertas vigentes. La gestión de stock, presupuesto, tickets y permisos pertenece al proyecto de producto; no todo ese alcance es necesario para el integrador.

B. El modelo

1. Dos candidatos para la misma tarea

Se compararon **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2** e **intfloat/multilingual-e5-small**. Ambos generan embeddings: vectores que permiten calcular similitud entre textos. MiniLM ofrece representaciones multilingües; E5 utiliza un encoder multilingüe y prefijos de entrada. La model card de E5 distingue tareas de recuperación y de similitud; el laboratorio de Clase 7 usa query para la consulta y passage para los productos. La elección del preprocesamiento forma parte del experimento y debe mantenerse documentada.

Las fichas de ambos modelos describen entrenamiento previo sobre textos y pares de ejemplos, no sobre precios actuales de supermercados argentinos. Los pesos se descargan de Hugging Face y la aplicación no los modifica. No se entrenó un modelo nuevo en el integrador: se realizó **inferencia** con parámetros ya aprendidos.

2. Resultados que sí están guardados

Método	Aciertos sobre 11	Lectura
Literal	4/11	Un positivo recuperado y tres rechazos correctos.
MiniLM	7/11	Rechaza los tres negativos, pero pierde cuatro positivos.
E5 small	8/11	Recupera los ocho positivos y acepta erróneamente los tres negativos.

Fuente: salidas guardadas en TP_Blanco_Griselda.ipynb, en la carpeta Clase 7. No se reentrenaron modelos ni se generaron métricas nuevas para redactar este informe. El PDF anterior confundía un acierto positivo literal con la exactitud global: 1/11 no era la exactitud de esa línea base.

La decisión de explorar E5 se apoya en la recuperación de positivos de esta prueba. No alcanza para concluir que sea mejor en general: son once consultas, sin partición independiente de validación. Ajustar un umbral sobre esos mismos casos puede sobreajustar la evaluación.

B. Errores y límites del modelo

Los errores explican más que un promedio

E5 clasifica “arroz integral” como yerba y “jabón para manos” como detergente; también asigna “té” a una familia presente en el catálogo. Estos son falsos positivos: el artículo solicitado no está disponible, pero el modelo elige el candidato más parecido. MiniLM rechaza “leche para el desayuno” y “milk” con el umbral utilizado: son falsos negativos dentro de la evaluación por familia.

El score coseno no es una probabilidad. Un umbral de 0,50 no tiene el mismo significado para todos los modelos. Los embeddings de E5 pueden producir valores elevados incluso en candidatos incorrectos; hace falta calibrar con negativos difíciles y un conjunto separado. Además, acertar la familia “leche” no demuestra que coincidan tipo, marca y litros.

Pipeline e InferenceClient

Con pipeline el modelo se ejecuta en el equipo: devuelve una lista anidada de vectores, que el código convierte a tensores para aplicar pooling y similitud. Con InferenceClient la solicitud viaja al proveedor remoto; sentence_similarity devuelve puntajes cuando el proveedor responde. La notebook muestra type() y el acceso por índice al resultado. No son el mismo tipo de contenido aunque ambas respuestas puedan representarse como listas.

La salida local guardada informa 0,90 s para MiniLM y 0,46 s para E5 en ese recorrido de evaluación; el temporizador comienza después de crear el scorer, por lo que no incluye la descarga ni toda la carga inicial. Son tiempos de ese equipo y esa muestra, no una promesa para el hosting.

La prueba remota devolvió **HTTP 402 Payment Required** por falta de créditos. Se conserva el error como evidencia; no hay scores ni latencia de inferencia remota completada para comparar. Los casos no ejecutados después del error no deben contarse como errores predictivos del modelo.

C. Dónde corre cada cosa

Dos versiones que conviene distinguir

Componente	Qué contiene y dónde se ejecuta
GitHub	Código, notebook y documentación del portfolio. Alojar código no ejecuta por sí mismo la app.
Hugging Face Hub	Pesos y model cards públicos. La primera carga local descarga esos archivos.
App académica Gradio	TP_Integrador/app.py: interfaz y ejecución de Transformers/PyTorch sobre el CSV. Necesita un proceso de servidor.
Clase 7: web y API	La copia de Clase 7 contiene un comparador E5 para ofertas guardadas. No es idéntica al backend modular de producto.
Producto en desarrollo	Android y web -> FastAPI -> estado compartido. La web local de 8002 usa actualmente reglas en el comparador; no debe atribuirse E5 a esa ruta.

Flujo de inferencia local: navegador -> servidor Gradio/Python -> tokenizer y modelo cargado en memoria -> vectores y similitud -> tabla devuelta al navegador. Los pesos llegan desde el Hub al primer uso; la consulta no necesita enviarse a Inference Providers en esta modalidad.

Flujo remoto ensayado: notebook -> InferenceClient -> proveedor de Hugging Face -> respuesta o error HTTP -> notebook. Ese recorrido quedó interrumpido por el error de créditos.

Se eligió inferencia local para evitar depender de créditos de la API. El costo de esa decisión es memoria y CPU en el servidor. La memoria del proceso completo con E5 y, si se habilita voz, Whisper, todavía no se midió en el hosting; el tamaño de los pesos no equivale al consumo total de RAM. No se dispone todavía de una medición del hosting que justifique afirmar que cabe en un plan gratuito.

D. El deploy

Publicar requiere algo más que un enlace al repositorio

La alternativa preparada para la app académica es un servicio Python administrado que ejecute Gradio. El README incluye un procedimiento para Render, pero no se verificó una URL pública final de esa app durante esta revisión. Por lo tanto, el deploy se declara **pendiente de acreditación**. Localhost y un túnel temporal que depende de la PC encendida no equivalen a un despliegue estable del comparador académico.

Alternativa	Ventaja	Motivo para evaluar o descartar
PaaS con Python	Despliegue del proceso web desde Git.	Candidata; RAM, arranque y persistencia aún sin medir en el hosting.
Hosting compartido	Administración sencilla de sitios.	El soporte de procesos Python persistentes y pesos de ML no está acreditado.
VPS	Control del sistema y dependencias.	Requiere administrar parches, HTTPS, procesos y copias de seguridad.
Serverless	Escalado por demanda.	La carga de modelos y la persistencia local requieren diseño adicional; evaluar arranques y almacenamiento.

El servidor debe estar disponible para recibir consultas, transformar entradas y ejecutar inferencia. Un servicio que escala a cero puede atenderlas tras un arranque; “encendido” describe disponibilidad del proceso de ejecución, no la obligación de pagar una VM permanente.

Firebase: Hosting puede servir el frontend con HTTPS, y Authentication puede gestionar correo y contraseña. El backend FastAPI necesita su propio entorno, por ejemplo Cloud Run. No se deben guardar contraseñas legibles en Firestore. Esta arquitectura está propuesta, no desplegada. Cambiar a Firebase requiere migración de identidades y verificación de tokens en el servidor.

E. Lo que se rompió

Error 1: inferencia remota sin créditos

Mensaje: “402 Payment Required” y “You have no remaining credits”.
Dónde: llamada a Hugging Face desde la notebook.
Causa: el proveedor no habilitó la inferencia con los créditos disponibles; un token válido no garantiza cuota.
Pasos: conservar la salida completa en la notebook, distinguir autenticación de cuota, no inventar resultados, ejecutar localmente con pipeline y documentar la limitación de la comparación remota.

Error 2: el producto existe, pero la búsqueda queda vacía

Mensaje observado: “No se encontraron ofertas coincidentes con precio. Probá con el nombre y la marca del producto”.
Dónde: web de desarrollo local.
Causa: la oferta “leche entera” tenía precio vacío y no informaba litros; además, la búsqueda literal no equiparaba 1 lt con 1000 ml.
Pasos: revisar datos sin modificarlos, reproducir la consulta, normalizar unidades, excluir tamaños distintos, advertir sobre presentación desconocida y explicar que debe cargarse un precio válido. Agregar pruebas de regresión para las tres situaciones. No inventar precios ni imágenes.

Límites de producto

Mil usuarios simultáneos requieren medir carga, limitar solicitudes, compartir cachés y reemplazar la persistencia local por servicios con concurrencia y respaldo adecuados. El estado JSON y los bloqueos de un único proceso no son una base distribuida. El OCR y la inferencia deben tener límites de tamaño y recursos. Los roles del hogar se comprueban en el backend; una interfaz que oculta botones no alcanza.

Si un proveedor cambia cuotas, mantener alternativas de inferencia y monitorear los errores permite degradar con un mensaje claro. El siguiente paso es unificar las versiones del código, separar datos reales de demostración y comprobar el recorrido completo desde otro dispositivo.

F. Vocabulario y entrega

Cinco líneas de TP_Integrador/app.py

Código	Concepto
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)	AutoTokenizer es una clase; tokenizer recibe la instancia creada por la carga.
model.eval()	eval es un método del modelo; configura el comportamiento de inferencia.
output = model(**tokens)	model es una instancia invocable; output recibe el resultado de la inferencia.
pooled = (output.last_hidden_state * attention_mask).sum(dim=1)	last_hidden_state es un atributo; sum es un método y dim recibe el argumento 1.
return F.normalize(pooled, p=2, dim=1)[0]	p es un parámetro y 2 su argumento. return entrega el vector normalizado.

Enlaces y fuentes

Portfolio: github.com/griseldablanco78-blip/Seminario2026 - rama Proyecto_Integrador
Carpeta académica: Clase 7/TP_Integrador.
URL de la app: pendiente de publicación y prueba externa.

Model cards: intfloat/multilingual-e5-small; paraphrase-multilingual-MiniLM-L12-v2.
Infraestructura: Firebase Authentication y Hosting con Cloud Run. Consulta: 25/10/2026.

Estado del trabajo: la prueba pública de la app, la medición de recursos del hosting y la presentación PPTX continúan pendientes. Los resultados documentados corresponden a la evidencia local disponible.